Multi-Container Runtime – TODO Solutions

Team Information:

Name: Adil Ahmed

SRN: PES2UG24AM013

Name: Abhyudaya M. Hegde

SRN: PES2UG24AM009

1. Multi-Container Runtime

The container runtime was implemented using the Linux `clone()` system call with multiple namespaces:

- PID namespace → isolates process IDs
- UTS namespace → allows separate hostname per container
- Mount namespace → isolates filesystem mounts

Each container is given a separate root filesystem using `chroot()`.

Code snippet:

```
int flags = CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS;
pid_t pid = clone(child_fn, stack + STACK_SIZE, flags | SIGCHLD,
&config);

Inside the child process:
chroot(cfg->rootfs);
chdir("/");
```

This ensures each container runs in an isolated environment.

2. CLI and IPC

A command-line interface (CLI) was implemented to interact with the supervisor.

Commands:

- start
- stop
- ps

Communication between CLI and supervisor is done using UNIX domain sockets.

Code snippet:

```
int sock = socket(AF_UNIX, SOCK_STREAM, 0);
connect(sock, (struct sockaddr *)&addr, sizeof(addr));

write(sock, &req, sizeof(req));
read(sock, &resp, sizeof(resp));
```

This allows sending structured requests and receiving responses.

3. Logging System

A producer-consumer model was used for logging container output.

- Producer thread reads from pipe (container stdout/stderr)
- Data is pushed into a bounded buffer
- Consumer thread writes logs to files

Code snippet (producer):

```
ssize_t n = read(fd, buffer, sizeof(buffer));
bounded_buffer_push(&ctx->log_buffer, &item);

Code snippet (consumer):

bounded_buffer_pop(&ctx->log_buffer, &item);
fprintf(log_file, "%s", item.data);
```

This design ensures non-blocking and efficient logging.

4. Kernel Monitor

A kernel module was implemented to monitor container memory usage.

- Uses ioctl interface for communication
- Tracks memory consumption per container
- Enforces:
    - Soft limits → warning
    - Hard limits → process termination

Code snippet:

```
ioctl(fd, REGISTER_CONTAINER, &config);

The kernel logs events using:

printk(KERN_INFO "Memory limit exceeded");
```

5. Scheduling Experiments

Scheduling behavior was tested using different nice values.

Example:

```
sudo ./engine start alpha ./rootfs-alpha /cpu_hog --nice 10
sudo ./engine start beta ./rootfs-beta /cpu_hog --nice -5
```

Observation:

- Lower nice value → higher priority
- Higher nice value → lower priority

This demonstrates Linux priority-based scheduling.

6. Cleanup

Proper cleanup mechanisms were implemented:

- Child processes are terminated using `kill()`
- Supervisor tracks container states
- Threads are cleaned up properly
- No zombie processes remain

Code snippet:

```
kill(container_pid, SIGKILL);
waitpid(container_pid, NULL, 0);
```

This ensures stable shutdown of containers.

Conclusion

This project demonstrates the implementation of a minimal container runtime using Linux system calls, IPC mechanisms, and kernel interaction. It provides insight into how container systems like Docker work internally.